Classe genérica para utilização no projeto de PCII

1. Introdução 1
2. Classe genérica Gclass 1
3. Código da classe Gclass 3
4. Código da classe Person 6
5. Criação da classe derivada Person 7
6. Exemplo: projeto 'generic_class' 8

1. Introdução

O objetivo da criação da classe genérica '**Gclass**' é o de apoiar a criação de classes no projeto de PCII, disponibilizando um conjunto de funcionalidades úteis e o suporte para o armazenamento da informação dos objetos da classe. As novas classes devem ser derivadas da classe 'Gclass' cuja descrição é feita a seguir.

2. Classe genérica Gclass

import sys
import datetime
import sqlite3
class Gclass:
	O módulo '**sys**' é usado para permitir a saída do programa em caso de erro. O módulo '**datetime**' é usado para identificação do atributo do tipo '**Date'**. O módulo '**sqlite3**' é usado para armazenar os objetos na Base de Dados. A seguir apresenta-se o exemplo de criação da classe derivada '**Person**'. A variável de classe '**obj**' é um dicionário cuja chave é o atributo identificador do objeto da classe e o valor, o respetivo objeto. A variável de classe '**lst**' é uma lista contendo os identificadores de todos os objetos da classe. A variável de classe '**pos**' indica a posição do objeto selecionado, que corresponde ao objeto cujo atributo identificador é o '**lst[pos]**'. A variável de classe '**sortkey**' armazena o atributo usado na ordenação dos objetos que é definido pelo método '**sort**'.
	from classes.gclass import Gclass
	import datetime
	class Person(Gclass):
	  # Dictionary of objects
	  obj = dict()
	  lst = list()
	  pos = 0
	  sortkey = ''
	  # Attribute names list, identifier attribute must be the first one and called 'id'
	  att = ['_id','_name','_dob','_salary']
	  # field description for use in, for example, input form
	  des = ['Id','Name','Date of Birth','Salary']
	  # Constructor: Called when an object is instantiated
	  def __init__(self, id, name, dob, salary):
	  super().__init__()
	  # Object attributes
	  id = Product.get_id(id)
	  self._id = id
	  self._name = name
	  self._dob = datetime.date.fromisoformat(dob)
	  self._salary = float(salary)
	  Person.obj[id] = self # Add the new object to the dictionary of objects
	  Person.lst.append(id) # Add the id to the list of object ids

def **__init__**(self): Construtor da classe genérica. Para já sem instruções. A criação dos atributos é feita nas classes derivadas.

def **from_string**(cls, str_data): - @classmethod. Construtor alternativo usado para criar um objeto com base num string com a informação dos seus atributos separados por ';'. Este é o formato utilizado nos ficheiros .csv o que permite criar os objetos diretamente a partir da informação contida nas linhas de um ficheiro .csv (atrib1;atrib2;…)

def **reset**(cls): - @classmethod. Faz o reset das variáveis de classe '**obj**', '**lst**' e '**pos**' ficando a classe sem objectos.

def **get_id**(cls, id): - @classmethod. Se 'id' maior que zero retorna 'id' sem alterações, se 'id' igual a zero retorna o valor máximo do 'id' dos objetos da classe adicionado de uma unidade. Isto permite gerar um identificador inteiro sequencial único para cada objeto da classe.

def **getlines**(cls, att, value): - @classmethod. Retorna uma sub-lista dos atributos identificadores 'lst' que contêm o attributo att = value.

Métodos para percorrer os objetos da classe com base na lista '**lst**'. Nesta lista é colocado o valor do atributo identificador quando o objeto é criado no método '**__init__**'. Por defeito, a ordem dos objetos é a ordem da sua criação mas esta sequência pode ser alterada pelo método '**sort(cls, att)**' que ordena a lista '**lst**' por ordem crescente de um dado atributo '**att**'.

---

def **nextrec**(cls): - Avança para o próximo objeto. Retorna o próximo objeto ou '**None**' se for o último.

def **previous**(cls): - Recua para o objeto anterior. Retorna o objeto anterior ou '**None**' se for o primeiro.

def **current**(cls, code=None): - Retorna o objeto corrente e/ou define o objeto com o atributo identificador '**code**' como o objeto corrente.

def **first**(cls): - Posiciona-se no início e retorna o primeiro objeto.

def **last**(cls): - Posiciona-se no fim e retorna o último objeto.

---

def **remove**(cls, p): - Remove o objeto cujo atributo identificador é '**p**' da lista '**lst**' e da Base de Dados.

def **insert**(cls, p): - Insere o objeto cujo atributo identificador é '**p**' na Base de Dados.

def **update**(cls, p): - Atualiza o objeto cujo atributo identificador é '**p**' na Base de Dados.

def **conv**(obj, att, value): - @staticmethod. Se o valor do atributo **att** é **str** ou **datetime** retorna com aspas.

def **sort**(cls, att, reverse): - Ordena '**lst**' os objetos por ordem crescente/decrescente do atributo '**att**' passado como argumento. Se '**reverse**' for omitido ou for '**False**' ordena por ordem crescente, se for '**True**' ordena por ordem decrescente.

def **find**(cls, value, att): - Retorna uma lista com os objetos '**obj**' que tenham o valor '**value**' igual ao valor do atributo cujo nome '**att**' é passado como argumento.

def **set_filter**(cls, f_dic = {}) - Redefine a lista '**lst**' apenas com os atributos identificadores dos objetos cujos atributos pertençam às listas definidas no dicionário '**f_dic**'. Por exemplo, no caso da classe pessoa, se pretendermos selecionar as pessoas que tenham o nome 'carlos' ou 'barbara' ou salario igual a '5000' ou '6000' devemos definir a variável f_dic = {'name':['carlos','barbara'], 'salary':[5000,6000]}. Para fazer o reset do filtro devemos chamar o método '**set_filter()**' sem parâmetros.

def **getatlist**(cls, att) - Retorna uma lista com os valores do atributo, cujo nome é '**att**', dos objetos da classe.

def **read**(cls, path = ''): - Lê os atributos e cria os objetos a partir da Base de Dados. Tem que existir uma tabela com o nome da cas

def **__str__**(self): - Método de instância que retorna a informação do objeto no formato (atrib1;atrib2;…). Este método é utilizado pelo método write(cls, path = '') para escrever a informação dos objetos para o ficheiro .csv.

3. Código da classe Gclass

```python
"""
@author: António Brito / Carlos Bragança
(2025) objective: Generic class
"""""
# Generic Class
import sys
import datetime
import sqlite3
class Gclass:
    # Constructor: Called when an object is instantiated
    def __init__(self):
        pass
##################################################
# generic code: no need to change for a new class
    # Class method to implement constructor overloading
    @classmethod
    def from_string(cls, str_data):
        str_list = str_data.split(";")
        strarg = 'cls(str_list[0]'
        for i in range(1, len(str_list)):
            strarg += ',str_list[' + str(i) + ']'
        strarg += ')'
        return eval(strarg)
    # Reset the class
    @classmethod
    def reset(cls):
        cls.obj = dict()
        cls.lst = list()
        cls.pos = 0
    # Object identifier auto increment
    @classmethod
    def get_id(cls, id):
        # Compute the auto increment
        id = int(id)
        if id == 0:
            if len(cls.lst) == 0:
                id = 1
            else:
                id = max(cls.lst) + 1
        return id
    # Class method to return the list of object id's having an attribute 'att' = 'value'
    @classmethod
    def getlines(cls, att, value):
        return [obj.id for obj in list(cls.obj.values()) if getattr(obj, att) == value]
    # Class methods to iterate (forward and backward) through the class objects
    @classmethod
    def nextrec(cls):
        cls.pos += 1
        return cls.current()
    @classmethod
    def previous(cls):
        cls.pos -= 1
        return cls.current()
```

```
54      @classmethod
55      def current(cls, code = None):
56          if code in cls.lst:
57              cls.pos = cls.lst.index(code)
58          if cls.pos < 0:
59              cls.pos = 0
60              return None
61          elif cls.pos >= len(cls.lst):
62              cls.pos = len(cls.lst) - 1
63              return None
64          else:
65              code = cls.lst[cls.pos]
66              return cls.obj[code]
67      @classmethod
68      def first(cls):
69          cls.pos = 0
70          return cls.current()
71      @classmethod
72      def last(cls):
73          cls.pos = len(cls.lst) - 1
74          return cls.current()
75      # Object delete method
76      @classmethod
77      def remove(cls, p):
78          obj = cls.obj[p]
79          id = cls.att[0][1:]
80          command = f'DELETE FROM {cls.__name__} WHERE {id}={cls.conv(obj,id,p)}'
81          cls.sqlexe(command)
82          cls.lst.remove(p)
83          del cls.obj[p]
84      # Object insert method
85      @classmethod
86      def insert(cls, p):
87          obj = cls.obj[p]
88          command = f'INSERT INTO {cls.__name__} VALUES('
89          for att in cls.att:
90              value = getattr(obj, att)
91              command += f'{cls.conv(obj, att, value)},'
92          command = command[:-1] + ")"
93          cls.sqlexe(command)
94      # Object update method
95      @classmethod
96      def update(cls, p):
97          obj = cls.obj[p]
98          command = f'UPDATE "{cls.__name__}" SET'
99          for att in cls.att:
100             value = getattr(obj, att)
101             command += f' {att[1:]} = {cls.conv(obj, att, value)},'
102             id = cls.att[0][1:]
103             command = command[:-1] + f' WHERE {id} = {cls.conv(obj, id, p)}'
104         cls.sqlexe(command)
105     @staticmethod
106     def conv(obj, att, value):
107         v = getattr(obj, att)
108         if type(v) == str or type(v) == datetime.date:
109             ret = f'"{value}"'
110         else:
111             ret = f'{value}'
112         return ret
113     # Sort objects by attribute class methods
114     @classmethod
115     def orderfunc(cls, e):
116         return getattr(cls.obj[e], cls.sortkey)
117     @classmethod
118     def sort(cls, att, reverse = False):
119         cls.sortkey = att
120         cls.lst.sort(key=cls.orderfunc, reverse= reverse)
121     # Find objects having an attribute equal to value
122     @classmethod
123     def find(cls, value, att):
124         lobj = cls.obj.values()
125         fobj = [obj for obj in lobj if getattr(obj, att) == value]
126         return fobj
```

```python
    # Apply a filter by attribute class methods
    @classmethod
    def set_filter(cls, f_dic = {}):
        if f_dic:
            code = cls.att[0]
            lobj = cls.obj.values()
            s = set()
            for att,listf in f_dic.items():
                s1 = set([getattr(obj, code) for obj in lobj if getattr(obj, att) in listf])
                s = s.union(s1)
            if len(s) > 0:
                cls.lst = list(s)
                cls.pos = 0
        else:
            obj = cls.current()
            cls.lst = list(cls.obj.keys())
            code = cls.att[0]
            cls.current(getattr(obj, code))
    # Get a list of objects attribute values
    @classmethod
    def getatlist(cls, att):
        return [getattr(obj, att) for obj in list(cls.obj.values())]
    # Read objects from db file
    @classmethod
    def read(cls, path = ''):
        cls.obj = dict()
        cls.lst = list()
        cls.path = path
        try:
            fh = open(path, 'r')
            fh.close()
            lista = cls.sqlexe("select * from " + cls.__name__)
            if lista != None:
                for r in lista:
                    objstr = f'{r[0]}'
                    for att in range(1,len(lista[0])):
                        objstr += f';{r[att]}'
                    cls.from_string(objstr)
        except FileNotFoundError:
            print(f"ERROR: {path} data file not found")
        except BaseException as err:
            print(f"Error in read method:\n{err}\n{type(err)}")
            sys.exit()
    # Instance method to obtain object info
    def __str__(self):
        strprint = "f'"
        for att in type(self).att:
            strprint += '{self.' + att + '};'
        strprint = strprint[:-1] + "'"
        return eval(strprint)
    # Execute a db query
    @classmethod
    def sqlexe(cls, command):
        resul = None
        try:
            con = sqlite3.connect(cls.path)
            cur = con.cursor()
            con.row_factory = sqlite3.Row
            tname = cls.__name__
            cur = con.execute(f"SELECT name FROM sqlite_master WHERE type='table' AND name='{tname}'")
            table = cur.fetchone()
            if table is None or table[0] != tname:
                print(f"ERROR: table {tname} missing in database {cls.path}")
                sys.exit()
            cur = con.execute(command)
            resul = cur.fetchall()
            con.commit()
            con.close()
        except sqlite3.Error as er:
            print(f'sqlite error: {er}')
        return resul
```

4. Código da classe Person

```python
"""
@author: António Brito / Carlos Bragança
(2025) objective: class Person
"""
# Class Person - generic version with inheritance
from classes.gclass import Gclass
import datetime
class Person(Gclass):
    obj = dict()
    lst = list()
    pos = 0
    sortkey = ''
    # Attribute names list, identifier attribute must be the first one and callled 'id'
    att = ['_id','_name','_dob','_salary']
    # Class header title
    header = 'Persons'
    # field description for use in, for example, input form
    des = ['Id','Name','Date of Birth','Salary']
    # Constructor: Called when an object is instantiated
    def __init__(self, id, name, dob, salary):
        super().__init__()
        # Object attributes
        id = Person.get_id(id)
        self._id = id
        self._name = name
        self._dob = datetime.date.fromisoformat(dob)
        self._salary = float(salary)
        # Add the new object to the dictionary of objects
        Person.obj[id] = self
        # Add the id to the list of object ids
        Person.lst.append(id)
    # id property getter method
    @property
    def id(self):
        return self._id
    @id.setter
    def id(self, id):
        self._id = id
    # name property getter method
    @property
    def name(self):
        return self._name
    @name.setter
    def name(self, name):
        self._name = name
    # dob property getter method
    @property
    def dob(self):
        return self._dob
    # dob property setter method
    @dob.setter
    def dob(self, dob):
        self._dob = dob
    # salary property getter method
    @property
    def salary(self):
        return self._salary
    # salary property setter method
    @salary.setter
    def salary(self, salary):
        self._salary = salary
    # age property getter method
    @property
    def age(self):
        tday = datetime.date.today()
        age = tday.year - self.dob.year
        if tday.month < self.dob.month or \
            (tday.month == self.dob.month and tday.day < self.dob.day):
            age -= 1
        return age
```

5. Criação da classe derivada Person

Como já referido os nomes das variáveis de instância que armazenam o valor das propriedades devem começar por '**_**' (underscore) e devem ser definidas propriedades '**@property**' para o caso do valor do atributo poder ser acedido (lido) e '**@atributo.setter**' para o caso do atributo poder ser alterado (escrito). O nome das propriedades deve ser o nome dos atributos sem o '**_**' (underscore).

Obrigatoriamente deve ser criada a variável de classe lista '**att**' (linha 14) com o nome das variáveis de instância da classe. Os nomes das variáveis de instância devem ser os definidos no '**__init__**' (linhas 24 a 27) e não o das propriedades. A variável de instância identificadora do objeto deve ser a primeira da lista '**att**' e deve ser designada de '_id'. O valor de 'id' é passado como parâmetro no método 'Person.get_id(id)' (linha 23) que retorna o valor de 'id' inalterado, se este for diferente de zero, ou se 'id' igual a zero retorna o novo valor igual ao valor máximo de 'id', dos objetos existentes, mais uma unidade, garantindo assim que cada objeto tem um valor único de identificação. Quando é criado um novo objeto, deve ser atribuído a 'id' o valor zero de forma a que seja gerado um valor inteiro sequencial único para cada um.

A variável de classe '**Person.obj**' é um dicionário que permite obter o objeto dado o valor do seu atributo identificador, neste caso o atributo '**id**' como definido na linha 29. Caso se pretenda aceder ao objeto '**Person**' cujo atributo identificador '**id**' é '**150**' utiliza-se a instrução '**Person.obj[150]**'. A variável de classe '**Person.lst**' permite percorrer os objetos da classe, utilizando os métodos '**nextrec,previous,current,first,last**' acima descritos ou diretamente através de um ciclo do tipo:

```
for id in Person.lst:
        print(Person.obj[id])
```

6. Exemplo: projeto 'classe_generica_gclass_lesson05'

O projeto 'classe_generica_gclass_lesson05' permite testar/demonstrar as funcionalidades da classe genérica através da classe derivada '**Person**' . O ficheiro '**ex_generic_class.py**' apresenta o seguinte menu ao utilizador.

```
---------------
l - list
b - beginning
n - next
p - previous
e - end
---------------
i - insert
m - modify
r - remove
---------------
s - sort by attribute
f - find by attribute
---------------
q - quit
---------------
600;Xavier;1960-04-21;5000.0
?
```

O utilizador deve colocar o cursor a seguir ao ponto de interrogação e digitar uma das opções do menu. Antes do menu é apresentada a lista dos objetos selecionados se a opção escolhida assim o determinar. A informação do objeto corrente é apresentada antes do ponto de interrogação. As opções existentes cobrem a maior parte das funcionalidades da classe genérica, podendo assim o utilizador testar as novas classes derivadas da classe '**Gclass**' antes de as utilizar num contexto mais complexo.